

LABORATÓRIO 5 — BI e Data Mining com Python

Disciplina: Gestão de Sistemas de Informação

Professor: Eduardo Max Amaro Amaral

Objetivo

Aplicar conceitos de OLAP (exploração multidimensional) e Mineração de Dados (regras de associação) utilizando Pandas sobre a nova base de 300 transações de vendas.

```
In [26]: import pandas as pd

print("PARTE 0 - CARGA DE DADOS")

df = pd.read_csv("base_vendas_transacional_300.csv")

# Normaliza a data mantendo apenas os 4 ultimos digitos do ano.
# Isso preserva compatibilidade com bases antigas e com a base nova.
partes_data = df["Data"].astype(str).str.split("/", expand=True)
df["Data"] = pd.to_datetime(
    partes_data[0] + "/" + partes_data[1] + "/" + partes_data[2].str[-4:],
    format="%d/%m/%Y",
    errors="coerce"
)

if df["Data"].isna().any():
    raise ValueError("Existem datas invalidas na nova base apos a normalizacao.")

display(df.head())

print("\nTotal de Registros:", len(df))
```

PARTE 0 - CARGA DE DADOS

	ID_Transacao	Data	Regional	Vendedor	Categoria	Produto	Valor_Venda
0	1	2026-03-21	Sul	Ana	Eletrônicos	Tablet	1800.0
1	1	2026-03-21	Sul	Ana	Acessórios	Capa Tablet	100.0
2	1	2026-03-21	Sul	Ana	Móveis	Cadeira Office	800.0
3	2	2026-03-03	Nordeste	Diego	Acessórios	Fone Bluetooth	200.0
4	3	2026-03-07	Sul	Elena	Móveis	Sofá 3 Lugares	2100.0

Total de Registros: 701

Parte 1 — OLAP (Exploração Multidimensional)

Nesta seção aplicaremos operações típicas de OLAP utilizando Pandas:

- Drill-down → detalhamento hierárquico
- Slice → filtro em uma dimensão
- Dice → recorte multidimensional

O objetivo é simular a navegação analítica sobre um cubo de dados.

```
In [27]: print("EXERCÍCIO 1.1 – DRILL-DOWN")

# Agregação por Regional
faturamento_regional = (
    df.groupby("Regional")["Valor_Venda"]
      .sum()
      .sort_values(ascending=False)
)

print("\nFaturamento por Regional:")
display(faturamento_regional)
```

EXERCÍCIO 1.1 – DRILL-DOWN

Faturamento por Regional:

Regional	
Sudeste	329300.0
Nordeste	328550.0
Sul	268150.0

Name: Valor_Venda, dtype: float64

```
In [28]: print("Drill-down da Regional Sudeste por Vendedor")

sudeste = df[df["Regional"] == "Sudeste"]

faturamento_vendedor = (
    sudeste.groupby("Vendedor")["Valor_Venda"]
      .sum()
      .sort_values(ascending=False)
)

display(faturamento_vendedor)
```

Drill-down da Regional Sudeste por Vendedor

Vendedor	
Carla	92350.0
Bruno	73650.0
Ana	70500.0
Diego	62550.0
Elena	30250.0

Name: Valor_Venda, dtype: float64

Exercício 1.2 — SLICE

O Slice consiste em fixar uma dimensão e analisar apenas um subconjunto específico.

Pergunta de Gestão: Qual foi o faturamento apenas da categoria "Eletrodomésticos"?

```
In [29]: print("EXERCÍCIO 1.2 – SLICE")

# Filtrando apenas a categoria Eletrodomésticos
slice_eleto = df[df["Categoria"] == "Eletrodomésticos"]

faturamento_slice = (
    slice_eleto.groupby("Regional")["Valor_Venda"]
    .sum()
    .sort_values(ascending=False)
)

print("\nFaturamento de Eletrodomésticos por Regional:")
display(faturamento_slice)
```

EXERCÍCIO 1.2 – SLICE

Faturamento de Eletrodomésticos por Regional:

Regional

Nordeste 84300.0

Sudeste 78700.0

Sul 62150.0

Name: Valor_Venda, dtype: float64

Exercício 1.3 — DICE

O Dice permite aplicar múltiplos filtros simultaneamente.

Pergunta de Gestão: Qual o faturamento da Regional Sudeste, considerando apenas a Categoria "Acessórios", e apenas os vendedores Ana e Carla?

```
In [30]: print("EXERCÍCIO 1.3 – DICE")

dice = df[
    (df["Regional"] == "Sudeste") &
    (df["Categoria"] == "Acessórios") &
    (df["Vendedor"].isin(["Ana", "Carla"]))
]

faturamento_dice = (
    dice.groupby("Vendedor")["Valor_Venda"]
    .sum()
    .sort_values(ascending=False)
)

print("\nFaturamento do recorte multidimensional:")
display(faturamento_dice)
```

EXERCÍCIO 1.3 – DICE

Faturamento do recorte multidimensional:

```
Vendedor
Carla      3150.0
Ana        2150.0
Name: Valor_Venda, dtype: float64
```

Parte 2 — Business Intelligence (Indicadores)

Nesta etapa iremos gerar KPIs estratégicos:

- Faturamento total
- Ticket médio
- Top vendedor
- Produto mais vendido
- Análise temporal (vendas por dia)

O objetivo é transformar dados em informação para tomada de decisão.

```
In [31]: print("PARTE 2 – KPI GERAL")

faturamento_total = df["Valor_Venda"].sum()
ticket_medio = df["Valor_Venda"].mean()

print(f"\nFaturamento Total: R$ {faturamento_total:,.2f}")
print(f"Ticket Médio: R$ {ticket_medio:,.2f}")
```

PARTE 2 – KPI GERAL

```
Faturamento Total: R$ 926,000.00
Ticket Médio: R$ 1,320.97
```

```
In [32]: print("\nEXERCÍCIO 2.2 – TOP VENDEDOR")

top_vendedor = (
    df.groupby("Vendedor")["Valor_Venda"]
      .sum()
      .sort_values(ascending=False)
)

display(top_vendedor.head(1))
```

EXERCÍCIO 2.2 – TOP VENDEDOR

```
Vendedor
Ana      225700.0
Name: Valor_Venda, dtype: float64
```

```
In [33]: print("\nEXERCÍCIO 2.3 – PRODUTO MAIS RENTÁVEL")

produto_top = (
    df.groupby("Produto")["Valor_Venda"]
      .sum()
      .sort_values(ascending=False)
)

display(produto_top.head(5))
```

EXERCÍCIO 2.3 – PRODUTO MAIS RENTÁVEL

Produto

Notebook 270000.0

Tablet 109800.0

Sofá 3 Lugares 100800.0

Monitor 24 66000.0

Máquina de Lavar 58000.0

Name: Valor_Venda, dtype: float64

```
In [34]: print("\nEXERCÍCIO 2.4 – VENDAS POR DATA")

# A coluna Data ja foi normalizada na carga inicial.

vendas_por_dia = (
    df.groupby("Data")["Valor_Venda"]
      .sum()
      .sort_index()
)

display(vendas_por_dia.head())
```

EXERCÍCIO 2.4 – VENDAS POR DATA

Data

2026-03-01 14950.0

2026-03-02 25450.0

2026-03-03 36200.0

2026-03-04 40200.0

2026-03-05 37000.0

Name: Valor_Venda, dtype: float64

Parte 3 — Data Mining

Nesta etapa buscamos padrões e relações nos dados.

Aplicaremos:

- Análise de correlação simples
- Tabela dinâmica (pivot table)
- Identificação de concentração de vendas

```
In [35]: print("EXERCÍCIO 3.1 – Padrão Regional x Categoria")

pivot_regional_categoria = pd.pivot_table(
    df,
    values="Valor_Venda",
    index="Regional",
    columns="Categoria",
    aggfunc="sum",
    fill_value=0
)

display(pivot_regional_categoria)
```

EXERCÍCIO 3.1 – Padrão Regional x Categoria

Categoria	Acessórios	Cozinha	Eletrodomésticos	Eletrônicos	Móveis
Regional					
Nordeste	10850.0	4800.0	84300.0	180300.0	48300.0
Sudeste	10800.0	3600.0	78700.0	179900.0	56300.0
Sul	8900.0	4200.0	62150.0	125600.0	67300.0

```
In [36]: print("\nEXERCÍCIO 3.2 – Concentração de Receita (Top Produtos)")

produto_receita = (
    df.groupby("Produto")["Valor_Venda"]
      .sum()
      .sort_values(ascending=False)
)

receita_total = produto_receita.sum()
receita_acumulada = produto_receita.cumsum() / receita_total * 100

concentracao = pd.DataFrame({
    "Receita": produto_receita,
    "% Acumulado": receita_acumulada
})

display(concentracao.head(10))
```

EXERCÍCIO 3.2 – Concentração de Receita (Top Produtos)

	Receita	% Acumulado
Produto		
Notebook	270000.0	29.157667
Tablet	109800.0	41.015119
Sofá 3 Lugares	100800.0	51.900648
Monitor 24	66000.0	59.028078
Máquina de Lavar	58000.0	65.291577
Geladeira	53200.0	71.036717
Ar Condicionado	46200.0	76.025918
Smartphone	40000.0	80.345572
Micro-ondas	35000.0	84.125270
Mesa de Jantar	25500.0	86.879050

```
In [37]: print("\nEXERCÍCIO 3.3 – Identificação de Outliers")

media = df["Valor_Venda"].mean()
desvio = df["Valor_Venda"].std()

limite_superior = media + 2 * desvio

outliers = df[df["Valor_Venda"] > limite_superior]
```

```
print(f"Média: {media:.2f}")
print(f"Desvio Padrão: {desvio:.2f}")
print(f"Limite Superior (2σ): {limite_superior:.2f}")
print(f"Quantidade de Outliers: {len(outliers)}")

display(outliers.head())
```

EXERCÍCIO 3.3 – Identificação de Outliers

Média: 1320.97

Desvio Padrão: 1293.88

Limite Superior (2σ): 3908.72

Quantidade de Outliers: 60

	ID_Transacao	Data	Regional	Vendedor	Categoria	Produto	Valor_Venda
20	10	2026-03-21	Sudeste	Bruno	Eletrônicos	Notebook	4500.0
27	12	2026-03-15	Sudeste	Carla	Eletrônicos	Notebook	4500.0
33	15	2026-03-05	Sudeste	Bruno	Eletrônicos	Notebook	4500.0
43	19	2026-03-18	Sul	Ana	Eletrônicos	Notebook	4500.0
53	22	2026-03-12	Sul	Elena	Eletrônicos	Notebook	4500.0

Conclusão Geral do Laboratório

Este laboratório demonstrou a aplicação prática de Business Intelligence e Data Mining com Pandas.

Principais aprendizados:

- OLAP permite navegação multidimensional (Drill-down, Slice, Dice)
- KPIs transformam dados brutos em informação estratégica
- Pivot tables revelam padrões regionais e setoriais
- Análise de concentração evidencia possível efeito Pareto
- Identificação de outliers auxilia na detecção de anomalias

A base de dados permitiu simular um cenário real de apoio à decisão empresarial.